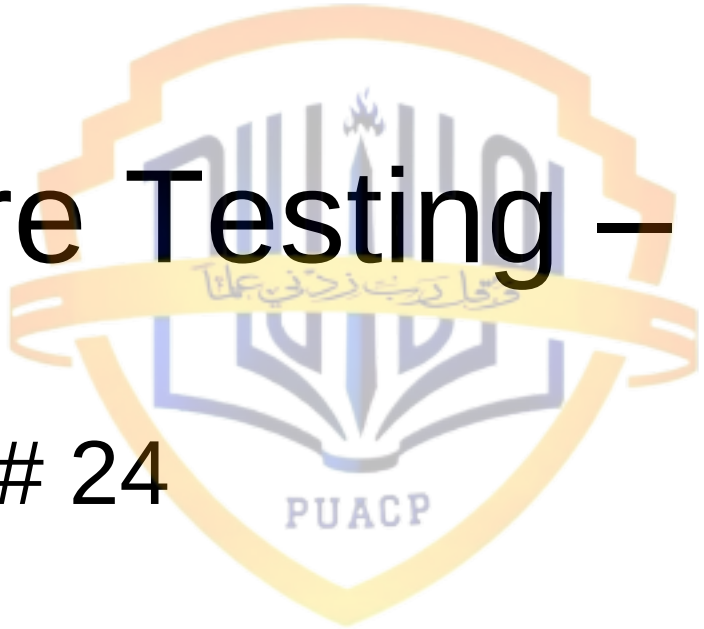
Software Testing – 2

Lecture # 24



Today's Lecture

- Today we'll continue our discussion on software testing
- We'll talk about testing objectives, testing principles, characteristics of testable software, and we'll end this lecture with an introduction of test case design

Testing Objectives

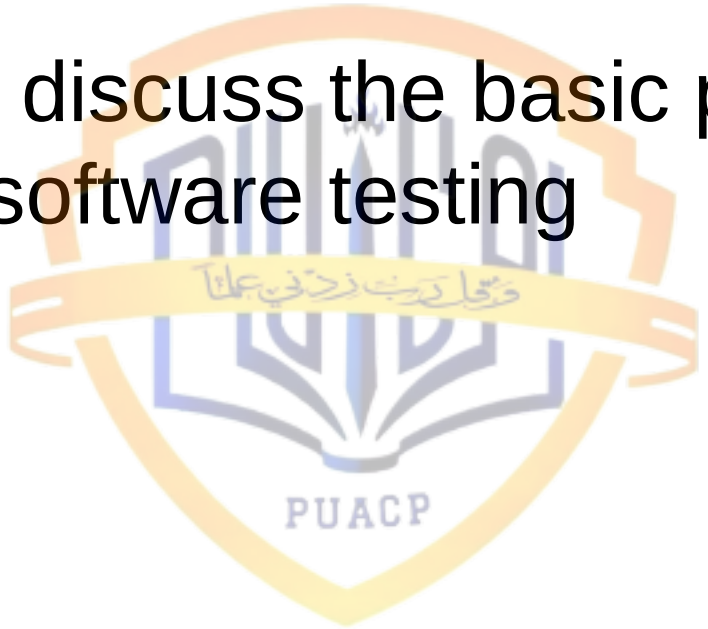
- Testing is a process of executing a program with the intent of finding an error
- A good test case is one that has a high probability of finding an as-yet-undiscovered error
- A successful test is one that uncovers an as- yet-undiscovered error

- These objectives imply a dramatic change in viewpoint. They move counter to the commonly held view that a successful test is one in which no errors are found. Our objective is to design tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort

- If testing is conducted successfully, it will uncover errors in the software, and as a secondary benefit, testing demonstrates that software functions appear to be working according to specification, that behavioral and performance requirements appear to have been met

- Data collected as a result of testing can provide a good indication of software reliability and some indication of the software quality as a whole
- Testing cannot show absence of errors and defects, it can show only that software errors and defects are present

- Let us now discuss the basic principles that guide software testing



Testing Principles - 1

- All tests should be traceable to customer requirements
  - The objective of software testing is to find defects. It follows that the most severe defects are those that cause systems to fail to meet their requirements
- Tests should be planned long before testing begins
  - Test planning can begin as soon as the requirements model is complete

Testing Principles - 2

- The Pareto (80-20) principle applies to software testing
  - 80% of all defects found will likely be traced to 20% of modules. These error-prone modules should be isolated and tested thoroughly
- The testing should begin “in the small” and progress toward testing “in the large”
  - The first tests planned and executed focus on individual components. As testing progresses, focus shifts to integrated clusters of components

Testing Principles - 3

- Exhaustive testing is not possible
  - The number of path permutations for even a moderately sized program is exceptionally large. It is impossible to execute every combination of paths during testing. It is possible, however, to adequately cover program logic and to ensure that all conditions in the component-level design have been exercised

Testing Principles - 4

- To be most effective, testing should be conducted by an independent third party
  - There are many testing techniques, as we will discuss them, and the software engineer who created the software system is not the best person to conducts all tests for the software. For certain kinds of testing, the best results are obtained by conducting these tests by professional testers

- We can say that software testing is a destructive process (psychologically speaking) and not a constructive process, because a software test will result in the breaking of software under test
- Programming on the other hand is a constructive process

- This change in thinking can help test planners and test designers to create effective test cases
- This concept also helps software designers and programmers to design and code software systems, which are testable. So, what is this testability?

Testability - 1

- Software testability is simply how easily [a computer program] can be tested
- Since testing is so difficult, it pays to know what can be done to streamline it
- It occurs as a result of good design. Data design, architecture, interfaces, and component-level detail can either facilitate testing or make it difficult

Testability - 2

- There are certain metrics that could be used to measure testability of a software product
- We can also call them the characteristics or attributes of testable software. These characteristics or attributes are important from quality assurance's point of view

Characteristics of Testable Software

- Operability
- Observability
- Controllability
- Decomposability
- Simplicity
- Stability
- Understandability

Operability

- “The better it works, the more efficiently it can be tested”
- The system has few bugs (bugs add analysis and reporting overhead to the test process)
- No bugs block the execution of tests
- The product evolves in functional stages (allows simultaneous development and testing)

Observability

- “What you see is what you test”
- Distinct output is generated for each input
- System states and variables are visible or queriable (e.g., transaction logs)
- All factors affecting the output are visible
- Incorrect output is easily identified
- Source code is accessible

Controllability - 1

- “The better we can control the software, the more testing can be automated and optimized”
- All possible outputs can be generated through some combination of input
- All code is executable through some combination of input

Controllability - 2

- Software and hardware states and variable can be controlled directly by the test engineer
- Input and output formats are consistent and structured
- Tests can be conveniently specified, automated, and reproduced

Decomposability

- "By controlling the scope of testing, we can more quickly isolate problems and perform smarter retesting"
- The software system is built from independent modules
- Software modules can be tested independently

Simplicity - 1

- “The less there is to test, the more quickly we can test it”
- Functional simplicity (e.g., the feature set is the minimum necessary to meet requirements)
- Structural simplicity (e.g., architecture is modularized to limit the propagation of faults)

Simplicity - 2

- Code simplicity (e.g., a coding standard is adopted for ease of inspection and maintenance)

Stability

- “The fewer the changes, the fewer the disruptions to testing”
- Changes to the software are infrequent
- Changes to the software are controlled
- Changes to the software do not invalidate existing tests
- The software recovers well from failures

Understandability - 1

- “The more information we have, the smarter we will test”
- The design is well understood
- Dependencies between internal, external, and shared components is well understood
- Changes to the design are communicated
- Technical documentation is instantly accessible

Understandability - 2

- Technical documentation is well organized
- Technical documentation is specified and detailed
- Technical documentation is accurate

Discussion of Testability - 1

- The characteristics we have discussed just are fundamental for software testers to efficiently and effectively perform software testing
- The all belong to the different developmental stages of software engineering process

Discussion of Testability - 2

- These characteristics clearly indicate that in order to have good software testing, best practices should be used in all activities of software engineering life cycle to develop software product, otherwise, we will end up with a product which is difficult to test

Attributes of a Good Test

- A good test has a high probability of finding an error
- A good test is not redundant
- A good test should be “best of breed”
- A good test should be neither too simple nor too complex

A good test has a high probability of finding an error

- The tester must understand the software and attempt to develop a mental picture of how the software might fail
- Ideally, the classes of failure are probed and a set of tests should be designed to show that software fails in a particular situation

A good test is not redundant

- Testing time and resources are limited and there is no point in conducting a test that has the same purpose as another test
- Every test should have a different purpose, even if it subtly different

A good test should be ‘best of breed’

- In a group of tests that have a similar intent, time and resource limitations may force us to execute only a subset of these tests. In such cases, the tests that has the highest likelihood of uncovering a whole class of errors should be used

A good test should be neither too simple nor too complex

- Ideally, each test should be executed separately, instead of being combined with many tests
- If tests are combined into one test, this can result in masking of errors

Test Case Design - 1

- The design of tests for software and other engineered products can be as challenging as the initial design of the product itself
- However, many software engineers treat testing as an afterthought, developing test cases that may “feel right” but have little assurance of being complete

Test Case Design - 2

- A rich variety of test case design methods have evolved for software, which provide the developer with a systematic approach to testing
- More important, methods provide a mechanism that can help to ensure the completeness of tests and provide the highest likelihood for uncovering errors in software

Test Case Design - 3

- Any engineered product can be tested in one of two ways
  - Knowing the specified function that a product has been designed to perform
  - Knowing the internal workings of a product

Test Case Design - 4

- In the first case, tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function
- In the second case, tests can be conducted to ensure that internal operations are performed according to the specifications and all internal components have been adequately exercised

Summary

- Today we've continued our discussion on software testing, and specifically we have talked about testing principles and characteristics of testable software products
- We'll start our discussion on test case design methods and testing techniques starting next lecture

References

- Software Engineering: A Practitioner's Approach 5th Edition by Roger S. Pressman (Chapter 17.1)